

pandas 데이터프레임 인덱서 loc, iloc

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5

```
In [3]: from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all' # 'last_expr'
```

데이터프레임의 기본 인덱싱

1. 열인덱싱

- 하나의 열만 선택 : `df['열이름']`
- 여러 개 열 선택 : `df[['열이름1', '열이름2', ...]]`

2. 행인덱싱 : 연속된 구간의 행데이터 선택(슬라이싱)

- `df['행_시작위치':'행_끝위치']`

3. 개별요소 접근 : 선택한 열에서 지정된 구간의 행데이터 선택

- `df['열이름']['행_시작위치':'행_끝위치']`
- `df['열이름']['시작_행이름':'끝_행이름']`

데이터프레임의 특별한 인덱서(indexer) : loc, iloc

- numpy와 같이 쉼표(,)를 사용하여 행과 열을 동시에 인덱싱하는 2차원 인덱싱
- [행 인덱스, 열 인덱스] 형식

- **loc** : 라벨값 기반의 2차원 인덱싱(명칭기반 인덱싱)
- **iloc** : 순서를 나타내는 정수 기반의 2차원 인덱싱(위치기반 인덱싱)

1. loc 인덱서

명칭기반 인덱서

1. `df.loc[행인덱싱 값]` # 행우선 인덱서
2. `df.loc[행인덱싱 값, 열인덱싱 값]`

인덱싱 값 유형

- 인덱스 데이터 : [index name] 또는 [index_name, column name]
- 인덱스 데이터 슬라이스
- 같은 행 인덱스를 갖는 불리언 시리즈(행 인덱싱인 경우)
 - 조건으로 추출 가능(불린 인덱싱)
- 위 값을 반환하는 함수

```
In [4]: df = pd.DataFrame(data=np.arange(10,22).reshape(3,4),
                        index=list('abc'),
```

```
columns=list('ABCD'))
df
```

```
Out[4]:
```

	A	B	C	D
a	10	11	12	13
b	14	15	16	17
c	18	19	20	21

```
In [5]: # 기본인덱싱 : 열 우선
df['A'] # 열인덱싱시리츠반환
df.C
df[['A','C']] # 데이터프레임반환
df['a':'b'] # 행인덱싱 -> 데이터프레임 반환
df['B']['c'] # 개별요소
```

```
Out[5]: a    10
b     14
c     18
Name: A, dtype: int64
```

```
Out[5]: a     12
b     16
c     20
Name: C, dtype: int64
```

```
Out[5]:
```

	A	C
a	10	12
b	14	16
c	18	20

```
Out[5]:
```

	A	B	C	D
a	10	11	12	13
b	14	15	16	17

```
Out[5]: np.int64(19)
```

1) 데이터프레임의 행 선택

형식 : `dataframe.loc['행인덱스값']`

```
In [6]: # 'a' 행 선택
df['a':'a'] # 데이터프레임 반환
df.loc['a'] # 시리즈 반환
```

```
Out[6]:
```

	A	B	C	D
a	10	11	12	13

```
Out[6]: A     10
B     11
C     12
D     13
Name: a, dtype: int64
```

주의. loc 인덱서에서는 열 단독 인덱싱은 불가능

```
In [8]: # df.loc['A'] # 행이름에 'A'가 존재하지 않으므로 KeyError 발생
```

2) 데이터프레임의 여러 행 선택

① 행인덱스 슬라이싱

형식: `dataFrame.loc['처음 행인덱스값' : '끝행인덱스값']`

- b행부터 c행의 모든 열 반환

```
In [9]: df['b':'c']
```

```
Out[9]:
```

	A	B	C	D
b	14	15	16	17
c	18	19	20	21

```
In [10]: df.loc['b':'c']
```

```
Out[10]:
```

	A	B	C	D
b	14	15	16	17
c	18	19	20	21

```
In [11]: df.loc[['b','c']]
```

```
Out[11]:
```

	A	B	C	D
b	14	15	16	17
c	18	19	20	21

행인덱스가 정수위치인덱스인 경우

```
In [12]: df2 = pd.DataFrame(data=np.arange(10,26).reshape(4,4),  
                           columns=list('abcd'))  
df2
```

```
Out[12]:
```

	a	b	c	d
0	10	11	12	13
1	14	15	16	17
2	18	19	20	21
3	22	23	24	25

값인덱싱 - 슬라이싱 초기값:끝값

```
In [13]: df2.loc[2]
```

```
Out[13]: a    18
         b    19
         c    20
         d    21
         Name: 2, dtype: int64
```

```
In [14]: df2.loc[2:3]
```

```
Out[14]:
```

	a	b	c	d
2	18	19	20	21
3	22	23	24	25

위치 인덱싱 - 슬라이싱 초기위치:끝위치 + 1

```
In [15]: df2.loc[2:4]
```

```
Out[15]:
```

	a	b	c	d
2	18	19	20	21
3	22	23	24	25

② 행인덱스 데이터를 리스트로 지정

형식: `dataFrame[[행이름1, 행이름2,... ]]`

- 여러 행 선택시 인덱서 데이터를 리스트로 사용
- 반환값이 데이터프레임이 됨

```
In [16]: df2.loc[[0,2]]
```

```
Out[16]:
```

	a	b	c	d
0	10	11	12	13
2	18	19	20	21

※ 참고. 데이터프레임 기본 인덱싱은 열기준

```
In [17]: df[['B', 'A']]
```

```
Out[17]:
```

	B	A
a	11	10
b	15	14
c	19	18

③ boolean indexing으로 행 선택

조건 식 수행

```
In [18]: df
```

```
Out[18]:
```

	A	B	C	D
a	10	11	12	13
b	14	15	16	17
c	18	19	20	21

- df의 A열의 값이 15보다 큰 행들 추출

```
In [19]: # 기본인덱싱
df['A'] > 15
df[df['A'] > 15]
```

```
Out[19]: a    False
b    False
c     True
Name: A, dtype: bool
```

```
Out[19]:
```

	A	B	C	D
c	18	19	20	21

```
In [20]: # loc인덱서 사용
df.loc[:, 'A'] > 15
df[df.loc[:, 'A'] > 15]
```

```
Out[20]: a    False
b    False
c     True
Name: A, dtype: bool
```

```
Out[20]:
```

	A	B	C	D
c	18	19	20	21

④ 인덱스 대신 인덱스 값을 반환하는 함수 사용

```
In [21]: def sel_row(df):
return df.A > 15
```

```
In [22]: sel_row(df)
```

```
Out[22]: a    False
b    False
c     True
Name: A, dtype: bool
```

```
In [23]: df[sel_row(df)]
```

```
Out[23]:
```

	A	B	C	D
c	18	19	20	21

```
In [24]: df.loc[sel_row(df)]
```

```
Out[24]:
```

	A	B	C	D
c	18	19	20	21

```
In [25]: df[lambda df: df.A > 15]
```

```
Out[25]:
```

	A	B	C	D
c	18	19	20	21

3) loc 인덱서로 열 선택

```
In [26]: df
```

```
Out[26]:
```

	A	B	C	D
a	10	11	12	13
b	14	15	16	17
c	18	19	20	21

```
In [27]: df.A  
df['A']  
df.loc[:, 'A']
```

```
Out[27]: a    10  
b    14  
c    18  
Name: A, dtype: int64
```

```
Out[27]: a    10  
b    14  
c    18  
Name: A, dtype: int64
```

```
Out[27]: a    10  
b    14  
c    18  
Name: A, dtype: int64
```

4) loc 인덱서로 개별 요소 선택

- 인덱싱으로 행과 열을 모두 받는 경우
- 형식 : **df.loc[행인덱스, 열인덱스]**
 - 라벨(문자열)인덱스 사용

```
In [28]: df2
```

```
Out[28]:
```

	a	b	c	d
0	10	11	12	13
1	14	15	16	17
2	18	19	20	21
3	22	23	24	25

```
In [31]: # 2행의 b열  
df2['b'][2:3] # 시리즈 반환  
df2.loc[2, 'b'] # 스칼라 반환
```

```
Out[31]: 2    19  
Name: b, dtype: int64
```

Out[31]: np.int64(19)

In [32]: df

Out[32]:

	A	B	C	D
a	10	11	12	13
b	14	15	16	17
c	18	19	20	21

In [39]: *# a행의 A열 값 수정*
df['A']['a':'a']
df['A']['a':'a'] = 100 # 기본인덱싱을 사용하는 경우 반환값이 시리즈(데이터프레임)이므로 스칼라
df.loc['a','A'] = 100 *# 개별요소의 값을 변경할 경우 loc인덱서를 사용하여 변경*
df

Out[39]: a 100
Name: A, dtype: int64

Out[39]:

	A	B	C	D
a	100	11	12	13
b	14	15	16	17
c	18	19	20	21

In [40]: *# a행 B열, c행 C열 모두 선택*
df.loc[['a','c'],['B','C']]

Out[40]:

	B	C
a	11	12
c	19	20

멀티인덱스를 갖는 데이터프레임

- 인덱스의 차원이 2차원 이상인 경우 : 다차원인덱스(멀티인덱스)

In [41]: df3 = pd.DataFrame(data=np.arange(10,22).reshape(3,4),
index=['a b c'.split()],
columns=['A','B','C','D'])
df3

Out[41]:

	A	B	C	D
a	10	11	12	13
b	14	15	16	17
c	18	19	20	21

In [42]: df3.index

Out[42]: MultiIndex([('a',),
('b',),
('c',)],
)

```
In [43]: ['a b c'.split()]
```

```
Out[43]: [['a', 'b', 'c']]
```

```
In [44]: df3.loc['a':'c']
```

```
Out[44]:
```

	A	B	C	D
a	10	11	12	13
b	14	15	16	17
c	18	19	20	21

```
In [45]: df4 = pd.DataFrame(data=np.arange(10,22).reshape(4,3),
                             index=[['a','a','b','b'],[1,2,1,2]],
                             columns=['A','B','C'])
df4
```

```
Out[45]:
```

		A	B	C
a	1	10	11	12
	2	13	14	15
b	1	16	17	18
	2	19	20	21

```
In [46]: df4.index
```

```
Out[46]: MultiIndex([(a, 1),
                     (a, 2),
                     (b, 1),
                     (b, 2)],
                    )
```

```
In [47]: df4.loc[('a', 1)]
df4.loc[(('a', 2), ('b', 1))]
```

```
Out[47]: A    10
         B    11
         C    12
         Name: (a, 1), dtype: int64
```

```
Out[47]:
```

		A	B	C
a	2	13	14	15
	1	16	17	18

2. iloc 인덱서

- 위치기반 인덱스
- 라벨(name)이 아닌 위치를 나타내는 정수 인덱스만 사용
- 위치 정수값은 0부터 시작
- 형식 : 데이터프레임.iloc[행, 열]

- 데이터프레임의 기본 인덱싱은 [행번호,열번호] 인덱싱 불가
- `iloc[]` 사용하면 가능
 - `iloc[행번호,열번호]`
 - `loc[행이름,열이름]`

In [48]: `df`

Out[48]:

	A	B	C	D
a	100	11	12	13
b	14	15	16	17
c	18	19	20	21

1) `iloc`을 이용하여 개별요소 선택

형식 : 데이터프레임.`iloc`[행번호,열번호]

결과값 : 스칼라값 반환됨

- 0행 1열 선택

In [49]: `df['B']['a':'a']`
`df.loc['a','B']`
`df.iloc[0,1]`

Out[49]: a 11
 Name: B, dtype: int64

Out[49]: `np.int64(11)`

Out[49]: `np.int64(11)`

2) `iloc`에 슬라이싱 적용

형식: `df.iloc`[시작인덱스:끝인덱스:슬라이싱간격]

- 1열의 0행,1행 선택 : 시리즈 형태로 반환

In [50]: `df.iloc[[0,1], 1]`

Out[50]: a 11
 b 15
 Name: B, dtype: int64

- 1열의 0행과 1행 선택 : 데이터프레임 형태로 반환

In [52]: `df.iloc[[0,1], 1:2]`
`df.iloc[0:2, 1:2]`

```
Out[52]:
```

	B
a	11
b	15

```
Out[52]:
```

	B
a	11
b	15

- 0행의 2번째 열부터 끝열까지 반환

```
In [53]: df.iloc[0, 2:]
```

```
Out[53]:
```

C	12
D	13

Name: a, dtype: int64

```
In [54]: df.iloc[0:1, 2:]
```

```
Out[54]:
```

	C	D
a	12	13

- 0행의 끝에서 두번째 열 이후까지 반환

```
In [55]: df.iloc[0, -2:]  
df.iloc[0:1, -2:]
```

```
Out[55]:
```

C	12
D	13

Name: a, dtype: int64

```
Out[55]:
```

	C	D
a	12	13

iloc인덱서 정리

- `iloc[행위치,열위치]` -> 원소값 반환
 - `iloc[행위치1:행위치2,열위치1:열위치2]` -> 원소 반환: df 반환
 - `iloc[행위치,열위치1:열위치2]` -> 원소반환 :시리즈 반환
 - `iloc[행위치1:행위치2,열위치]` -> 원소반환 : 시리즈 반환
-